

دليل هندسي عملي - من تحليل الشركة إلى وكيل موثوق

هندسة الوكلاء الذكيين للشركات

منهج قابل لإعادة الاستخدام لبناء وكيل خدمة عملاء، مبيعات، حجوزات، دعم فني، معرفة داخلية، أو عمليات - مع عقود واضحة وحماية واختبارات.

ماذا ستتعلم؟

- كيف تحلل العمل وتجهز البيئة قبل كتابة الكود
- كيف تفصل المصنف وال Guard والسياسات والحماية
- كيف تضيف الذاكرة والمعرفة والأدوات والتذاكر بأمان
- كيف تختبر الحوار وتمنع تثبيت الأخطاء داخل الاختبارات

كيف تستخدم هذا الدليل

مرجع قرار، لا وصفة نسخ ولصق

هذا الدليل لا يفترض أن كل وكيل هو Chatbot. الوكيل المؤسسي نظام قرار: يفهم الرسالة ضمن عقد منظم، يتحقق من الفهم، يطبق سياسة العمل، يستخدم معرفة أو أداة مصرحاً بها، يحفظ الحالة، ثم يشرح النتيجة. يمكنك قراءة الدليل بالترتيب عند بناء مشروع جديد، أو الرجوع إلى فصل محدد عند إضافة خاصية.

الفكرة المركزية

النموذج يقترح الفهم، وال Guard يتحقق منه، وال Policy يقرر، وال Workflow ينسق، وال Response Builder يشرح. لا تسمح لطبقة واحدة أن تقوم بكل هذه الوظائف.

خريطة أجزاء الدليل

الجزء	العنوان	ما الذي يبينه
1	قبل كتابة الكود	تحويل احتياج الشركة إلى نطاق وعقود وبيئة قابلة للقياس قبل اختيار النموذج أو الإطار.
2	نواة الذكاء الحواري	بناء الحالة والمصنف وال Guard وجمع المتطلبات والسياسات والردود كسلسلة منفصلة المسؤوليات.
3	المعرفة والأفعال والبيانات	إضافة RAG والوكلاء المتخصصين والأدوات والتذاكر والتخزين دون ربط منطق العمل بمزود واحد.
4	الموثوقية والأمان والاختبار	التعامل مع الأعطال والحماية والصلاحيات والتقييم حتى يصبح السلوك قابلاً للإثبات لا مجرد عرض تجريبي.
5	إعادة الاستخدام والتوسع	وصفات جاهزة لإضافة الخصائص وتكييف الهيكل لوكلاء شركات ومهام مختلفة وخطة تنفيذ تدريجية.

1

قبل كتابة الكود

تحويل احتياج الشركة إلى نطاق وعقود وبيئة قابلة للقياس قبل اختيار النموذج أو الإطار.

1

ابدأ بقرار العمل لا بال Prompt

ما الذي سيغيره الوكيل في الواقع؟

أول سؤال ليس: أي نموذج سأستخدم؟ السؤال الصحيح: ما القرار أو العملية التي تريد الشركة تحسينها؟ قد يكون الهدف تقليل زمن الرد، جمع طلب مكتمل، حجز موعد، البحث في سياسات داخلية، أو تنفيذ إجراء بعد موافقة. إذا لم يكن الناتج محددًا فلن تعرف ما الذي تختبره.

بطاقة تعريف الوكيل

مثال	السؤال الذي تجيب عنه	العنصر
عميل محتمل أو موظف دعم	من يتحدث مع الوكيل؟	المستخدم
تذكرة كاملة أو حجز مؤكد	ما الناتج القابل للقياس؟	النتيجة
شرح وجمع متطلبات وإنشاء طلب	ما الذي يستطيع فعله؟	النطاق
لا يحدد سعرا نهائيا ولا ينفذ إجراء حساسا دون موافقة	ما الذي لا يستطيع فعله؟	الحدود
كتالوج خدمات وسياسات وقاعدة معرفة	من أين تأتي المعلومات؟	مصدر الحقيقة
سؤال توضيحي أو تحويل صادق لمسار بشري متاح	ماذا يحدث عند الشك؟	مسار الفشل

اختبار مبكر

اكتب عشر محادثات واقعية قبل أي كود. إذا لم تستطع تحديد الرد الصحيح لكل واحدة، فالمشكلة في تعريف العمل وليست في النموذج.

مخرجات هذه المرحلة

- جملة هدف واحدة يمكن قياسها.
- قائمة بما يسمح للوكيل فعله وما يمنع عنه.
- أمثلة نجاح وفشل وحدود واضحة.
- مالك عمل يوافق على القرارات والسياسات.

2

ورشة اكتشاف الشركة

اجمع النظام الحقيقي قبل تحويله إلى كود
ابن الوكيل من مقابلات قصيرة مع أصحاب العمل، لا من تخمينات المطور. اجمع الخدمات والأقسام والأسئلة المتكررة والمستندات والأنظمة الخارجية والسياسات والاستثناءات. كل معلومة غير محسومة تتحول لاحقا إلى هلوسة أو شرط مبعر داخل الكود.

قائمة الأسئلة

- ما الخدمات أو العمليات التي يغطيها الوكيل؟
- ما المتطلبات الإلزامية لكل خدمة؟ وما نوع كل قيمة؟
- أي قسم يملك الطلب؟ وهل توجد أقسام داعمة؟
- ما المعلومات العامة، وما المعلومات المقيدة حسب الدور؟
- ما الإجراءات التي تحتاج تأكيدا أو موافقة بشرية؟
- ما قواعد السعر والوعود والمواعيد وسياسات الإلغاء؟
- ما الأنظمة المطلوب الربط معها؟ ومن يملك مفاتيحها؟
- كيف يعرف الموظف أن الوكيل أنجز المهمة بنجاح؟

حول الإجابات إلى ملفات بيانات

لماذا لا نضع في Prompt فقط؟	المكان المقترح	المعلومة
لأن التطبيق يحتاجها للتحقق والعرض والاختبار	JSON أو domain/service_catalog.py مضبوط	الخدمات والمتطلبات
حتى يمكن الاسترجاع والتوثيق والتحديث	أو مخزن وثائق knowledge/*.json	المعرفة
لفصل صلاحيات كل وكيل متخصص	skills/*.json	مهارات الأقسام
لأنها قرارات عمل حتمية قابلة للاختبار	application/policy_engine.py	السياسات

علامة خطر

إذا قال أحدهم: دع النموذج يقرر كل شيء، أسأله من يتحمل الخطأ عندما يخترع سعرا أو ينفذ إجراء غير مصرح. النموذج جزء من النظام وليس نظام الحوكمة.

3

العقود ومقاييس النجاح

حول اللغة العامة إلى بيانات يمكن اختبارها

كل حدود بين طبقتين تحتاج عقدا واضحا. بدل أن يعيد المصنف نصا حرا، اجعله يعيد نموذجا منظما: `intent`، `service candidates`، `requirements`، `evidence`، `confidence`. وتسمح `Guard` بفحصها.

مثال عقد قرار الرسالة

```
class MessageDecision(BaseModel):
    primary_intent: MessageIntent
    service_candidates: list[ServiceCandidate] = []
    extracted_requirements: list[ExtractedRequirement] = []
    pricing_requested: bool = False
    needs_clarification: bool = False
    confidence: float = Field(ge=0.0, le=1.0)
```

مقاييس لا تعتمد على الانطباع

طريقة الحساب	ما الذي يقيسه	المقياس
عدد الرسائل الصحيحة / مجموعة الاختبار	اختيار الخدمة والقسم	دقة التوجيه
الحقول الصحيحة مقابل المتوقعة	جمع الحقول المطلوبة بلا اختراع	اكتمال المتطلبات
محاولات التوضيح لكل جلسة	كم مرة احتاج العميل سؤالاً إضافياً	معدل التوضيح
مصادر صحيحة وعدم قبول مصدر مخترع	اعتماد الإجابة على مصادر مسترجعة	Grounding
تنفيذ واحد صحيح مع idempotency	نتيجة أداة أو تذكيرة أو حجز	نجاح الإجراء
لكل عقدة P95 و P50	تجربة المستخدم	زمن الاستجابة

قاعدة

لا تجعل `confidence` وحده بوابة الأمان. الثقة رقم من النموذج؛ الدليل والسياسة والصلاحيية تحقق مستقل.

4

تجهيز البيئة والمتطلبات

بيئة بسيطة أولاً ثم خدمات إضافية عند الحاجة
 ابدأ بأقل بيئة تحقق دورة كاملة. Python و FastAPI و Pydantic تكفي للنواة. أضف PostgreSQL عندما تحتاج حفظاً دائماً، و Redis عندما تحتاج حدود طلبات موزعة، وواجهة ويب عندما يصبح الـ API مثبتاً. Docker و CI مفيدان لاحقاً لكنهما ليسا شرطاً لفهم التصميم.

الحد الأدنى المقترح

وظيفته	الخيار	الفئة
النطاق والتطبيق والاختبارات	Python 3.12+	لغة
عقد HTTP والتبعيات والأخطاء	FastAPI	API
نماذج الحالة والقرارات والإعدادات	Pydantic v2	عقود
حالة وعقد ومسارات واضحة	أو منسق مخصص LangGraph	تدفق
تبدل المزود دون تغيير منطق العمل	مزود عبر Adapter	نموذج
وحدة وتكامل وانحدار حوار	pytest	اختبارات
جلسات وتذاكر وتدقيق	PostgreSQL	تخزين اختياري
حدود طلبات مشتركة بين النسخ	Redis	حماية اختيارية

بداية محلية نموذجية

```
python -m venv .venv
# Windows PowerShell
.venv\Scripts\Activate.ps1

python -m pip install -e ".[dev]"
python -m pytest -q
python -m uvicorn company_agent.main:app --reload
```

ملف الإعدادات

- ضع القيم العامة في Settings مع أنواع وحدود واضحة.
- ضع الأسرار في env. محلياً ولا ترسلها إلى Git.
- اجعل model name و base URL و timeouts إعدادات لا ثوابت.
- امنع تكرار الحقول داخل Settings لأن التعريف الأخير يخفي السابق.
- أنشئ env.example بأسماء المتغيرات وقيم وهمية فقط.

قبل البدء

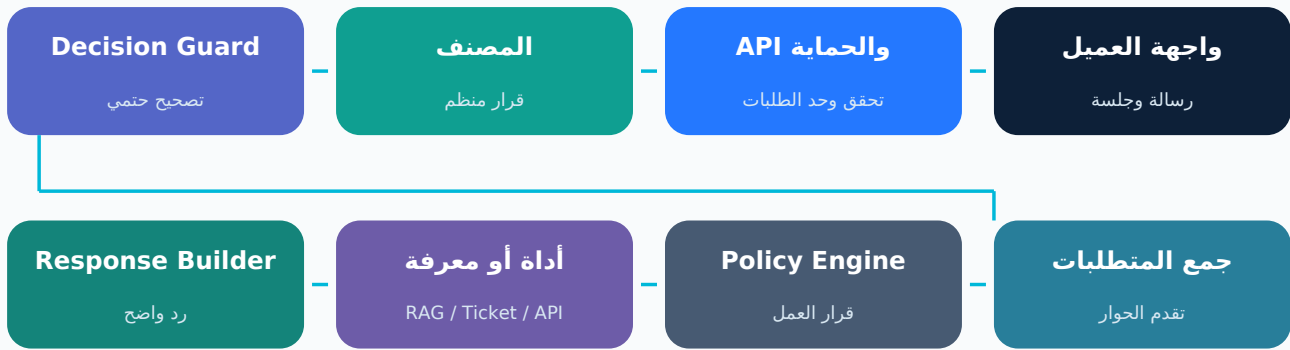
اكتب أمرا واحدا يثبت أن الإعدادات تستورد، وأمرا يختبر اتصال كل خدمة خارجية. التشخيص المبكر أرخص من ملاحقة 503 داخل الواجهة.

5

الهيكل والحدود بين الطبقات

اجعل اتجاه الاعتماد نحو الداخل

الهيكل الجيد يسمح بتبديل قاعدة البيانات أو النموذج أو الواجهة دون تغيير قواعد العمل. المجال Domain لا يعرف. تنفذ هذه العقود Infrastructure البنية. Ports. يستخدم عقود Application التطبيق. PostgreSQL أو FastAPI. الرسم Workflow ينسق ولا يحتوي تفاصيل كل قرار.



هيكل مرجعي قابل للتقليل أو التوسع

```

src/company_agent/
  api/
    routes.py
    schemas.py
    dependencies.py
  application/
    message_classifier.py
    decision_guard.py
    requirements_collector.py
    policy_engine.py
    response_builder.py
  ports/
  domain/
    conversation_state.py
    message_decision.py
    service_catalog.py
    ticket.py
  graph/
    state.py
    workflow.py
  infrastructure/
    model.py
    repositories.py
    traffic_guard.py
    knowledge_base.py
  prompts/
  knowledge/
  skills/
  tests/
  
```

الطبقة	تحتوي	لا تحتوي
Domain	ونماذج وقواعد نقيّة Enums	أو استدعاء نموذج SQL أو HTTP
Application	Ports وسياسات و Guards و Use cases	تفاصيل مزود أو إطار واجهة
Infrastructure	ونموذج Redis وقواعد بيانات و Adapters	قرار تجاري مبعثر
API	تحقق HTTP وربط تبعيات وترجمة أخطاء	منطق محادثة كامل
Graph	ترتيب العقد والمسارات	تنفيذ ضخم داخل عقدة واحدة

2

نواة الذكاء الحواري

بناء الحالة والمصنف وال Guard وجمع المتطلبات والسياسات والردود كسلسلة منفصلة المسؤوليات.

6

حالة المحادثة: ذاكرة العمل الرسمية

لا تجعل سجل الرسائل هو المصدر الوحيد للحقيقة

تمثل ما يعرفه التطبيق حالياً: المرحلة والخدمة والمتطلبات والسؤال المنتظر والمسودة ConversationState والعدادات. سجل الرسائل مفيد للسياق، لكنه نص غير منظم. الحالة المنظمة تسمح باستئناف الجلسة واختبار الانتقالات ومنع تكرار الأفعال.

الحد الأدنى لحالة محادثة متعددة الرسائل

```
class ConversationState(BaseModel):
    session_id: str
    revision: int = 0
    stage: ConversationStage = ConversationStage.NEW
    current_service: ServiceKey | None = None
    collected_requirements: dict[str, str] = {}
    missing_requirements: list[str] = []
    last_question_key: str | None = None
    ticket_draft: TicketDraft | None = None
    created_ticket: Ticket | None = None
    counters: ConversationCounters = ConversationCounters()
```

مبادئ تصميم الحالة

- كل حقل له معنى واحد، ولا تستخدم last_message لاشتقاق كل شيء كل مرة.
- المرحلة Enum لا نص حر: discovery ثم requirements ثم draft_review ثم created.
- احفظ السؤال النشط بمفتاحه حتى تعرف ماذا تعني الإجابة القصيرة.
- استخدم revision لمنع طلبين متزامنين من الكتابة فوق بعضهما.
- لا تخزن أسراراً أو بيانات شخصية غير لازمة داخل الحالة.
- عرف قواعد واضحة لمسح العدادات عند حدوث تقدم حقيقي.

مقاطعة مفيدة

إذا سألت العميل: ما اسمك؟ أثناء جمع المتطلبات، أجب عن السؤال دون مسح current_service أو last_question_key. الرسالة التالية يجب أن تكمل من النقطة نفسها.

7

المصنف: تحويل الرسالة إلى عقد

لا يرد على العميل داخل هذه الطبقة LLM
المصنف يقرأ الرسالة وسياقا مختصرا، ثم يعيد JSON مطابقا لعقد MessageDecision. لا يكتب جوابا للعميل ولا ينفذ أداة. مهمته اقتراح النية والخدمة والحقائق الصريحة فقط.

افصل ثلاثة أشياء

مثال	السؤال	المفهوم
clarification أو service_request	ماذا يفعل العميل بهذه الرسالة؟	Intent
website_development	أي مجال عمل أقرب؟	Service
اللغتين العربية والإنجليزية = languages	ما الحقيقة التي صرح بها؟	Requirement

ولا يخلط منطق العمل Runnable يعيد Builder

```
def build_message_classifier(model: BaseChatModel) -> Runnable:
    parser = PydanticOutputParser(pydantic_object=MessageDecision)
    prompt = PROMPT.partial(
        format_instructions=parser.get_format_instructions(),
        catalog_context=catalog_prompt_context(),
    )
    return prompt | model.bind(
        response_format={"type": "json_object"}
    ) | parser
```

سياق المصنف

- المرحلة الحالية والخدمة الحالية.
- مفاتيح المتطلبات المجموعة والمفقودة، لا كل أسرار الحالة.
- السؤال النشط ونوع قيمته.
- وجود مسودة وإصدارها دون نسخ محتوى ضخم بلا داع.
- ملخص قصير لآخر رد عند الحاجة لفهم المتابعة.

لا تثق بالمصنف مباشرة

حتى مع Structured Output قد يعيد النموذج دليلا غير موجود أو ينقل معلومة من السياق السابق. لذلك تأتي طبقة Decision Guard دائما بعده.

8

تصحيح قرارات النموذج: Decision Guard

طبقة حتمية نقية بين الذكاء الاصطناعي ومنطق العمل

أنشئ application/decision_guard.py عندما يؤثر قرار النموذج في حالة أو صلاحية أو إجراء. يأخذ القرار الخام والرسالة والحالة، ثم يعيد نسخة مصححة. لا يستدعي نموذجاً ولا قاعدة بيانات ولا Redis ولا يبني الرد.

ماذا يفعل؟

- من الرسالة الحالية evidence لا يملك requirement يحذف كل Grounding:
- يصحح حالات واضحة مثل لم أفهم أو ما اسمك Intent repair:
- يكتشف سعراً صريحاً حتى لو أخطأ النموذج Policy signals:
- إلا مع مسودة وفعل تعديل واضح ticket_edit لا يحول رسالة إلى State-aware safety:
- يعالج الهمزات والمسافات والتشكيل واللهجات المحددة Normalization:

قالب Guard يمكن توسيعه بقواعد صغيرة

```
def guard_message_decision(conversation, decision, message):
    guarded = decision.model_copy(deep=True)

    guarded.extracted_requirements = [
        item for item in guarded.extracted_requirements
        if evidence_exists(item.evidence, message)
    ]

    if message_expresses_confusion(conversation, message):
        guarded.primary_intent = MessageIntent.CLARIFICATION
        guarded.extracted_requirements = []
        guarded.needs_clarification = True
        return guarded

    if message_is_identity_question(message):
        guarded.primary_intent = MessageIntent.GREETING
        guarded.service_candidates = []
        return guarded

    if has_draft(conversation) and explicit_edit_signal(message):
        guarded.primary_intent = MessageIntent.TICKET_EDIT

    return guarded
```

ما الذي لا يوضع فيه؟

المكان الصحيح	السبب	لا تضع
مستقل validator أو message_classifier	يفقد الحتمية ويضاعف التكلفة	استدعاء LLM
service غير repository	يحول التحقق إلى عملية ذات أثر	حفظ قاعدة بيانات
traffic_guard	يحتاج زمناً وRedis	Rate limit
response_builder	يخلط القرار مع العرض	نص الرد

اختبار كل قاعدة

اكتب Positive case و Negative case. مثال: لم أفهم تصيح clarification، لكن أريد إضافة ميزة لا يجب أن تصيح edit. أما ما اسمك فلا تصيح edit، أثناء المسودة، أضف ميزة تصيح clarification.

كتالوج الخدمات وجمع المتطلبات

حول رحلة السؤال إلى بيانات قابلة للمراجعة

الاسم، القسم، الوصف، والمتطلبات، service key: هو تعريف الشركة داخل النظام service_catalog.py. لا يخلع؛ يطبق الحقائق المقبولة على الحالة ويحدد أول متطلب إلزامي مفقود requirements_collector.py.

عقد خدمة مستقل عن النموذج

```
class RequirementDefinition(BaseModel):
    key: str
    label_ar: str
    question_ar: str
    required: bool = True
    value_kind: RequirementValueKind = RequirementValueKind.TEXT

class ServiceDefinition(BaseModel):
    key: ServiceKey
    primary_department: Department
    description_ar: str
    requirements: tuple[RequirementDefinition, ...]
```

خوارزمية Collector

- 1 **اختيار الخدمة**
أفضل مرشح صالح أو الخدمة الحالية
- 2 **تطبيع المفاتيح**
تحويل aliases إلى المفاتيح الرسمية
- 3 **تخزين الحقائق**
فقط بعد confidence وGuard
- 4 **حساب النواقص**
وغير موجود required
- 5 **تحديد المرحلة**
requirements أو draft_review

الإجابة القصيرة

الرسالة داخل الموقع لا تعني شيئاً وحدها. معناها يأتي من last_question_key = deployment_channel. لذلك مرر للمصنف واحفظه في الحالة active requirement.

أنواع القيم

النوع	مثال	طريقة التحقق والعرض
TEXT	هدف المشروع	نص مختصر غير فارغ
BOOLEAN	موقع حالي؟	ثم عرض عربي مفهوم yes/no
HUMAN_LANGUAGES	العربية والإنجليزية	منع خلط لغة البرمجة بلغة الواجهة
FEATURE_LIST	دفع وحجوزات	مميزات متعددة مع إزالة التكرار

فهم تكرار المعنى ومحاولات التوضيح

إشارة حوارية وليست حكما أمنيا

أنشئ منطق التكرار في policy_engine.py أو dialogue_policy.py عندما تحتاج تغيير الشرح أو معرفة أن المستخدم لم يتقدم. لا تسمى العداد rapid ما لم تقيس الزمن. TrafficGuard وحده مسؤول عن كثافة الطلبات.

افصل العدادات

العداد	متى يزيد	ماذا يسبب
exact_repeat_count	النص المنطوق مطابق لآخر رسالة	اختيار صياغة أخرى فقط
semantic_repeat_count	النية والخدمة والهدف متقاربة	شرح مختلف أو مثال
clarification_attempts	تصريح حقيقي بعدم الفهم بلا متطلب جديد	تبسيط ثم مسار مساعدة
rate violations	طلبات كثيرة داخل نافذة زمنية	429 أو تعليق مؤقت عبر TrafficGuard

التقدم الحقيقي يعيد ضبط العدادات

```
def track_semantic_repetition(state, decision):
    has_progress = bool(decision.extracted_requirements) or (
        decision.primary_intent in {
            MessageIntent.TICKET_EDIT,
            MessageIntent.TICKET_CONFIRM,
        }
    )
    is_confused = (
        decision.primary_intent is MessageIntent.CLARIFICATION
        and not decision.extracted_requirements
    )

    if has_progress:
        reset_dialogue_counters(state)
    elif same_semantic_signature(state, decision):
        state.counters.semantic_repeat_count += 1

    if is_confused:
        state.counters.clarification_attempts += 1
```

Semantic signature

ابن توقيعاً صغيراً من family + service + canonical goal. استخدم تطبيعاً بسيطاً أو embeddings عند الحاجة، لكن لا تجعل التشابه وحده ينفذ حظراً. في أسئلة السعر يمكن اعتبار كل الصياغات ضمن family = للخدمة نفسها pricing.

الخطأ الشائع

لا تزد exact و semantic و rapid و clarification معاً عند تكرار النص. الرسالة المكررة قد تكون مشكلة وإجهة، أو عدم فهم، أو إعادة إرسال؛ لا تعني هجوماً.

11

قرار العمل الحتمي: Policy Engine

يفصل ما يجوز فعله عن طريقة صياغة الرد

أنشئ application/policy_engine.py عندما توجد قواعد لا تريد تركها للنموذج: السعر، الموافقات، التكرار الحواري، التحويل، أو السماح بإنشاء تذكرة. يعيد PolicyDecision ولا يكتب رسالة طويلة.

اجعل نتيجة السياسة قابلة للسجل والاختبار

```
class PolicyDecision(BaseModel):
    action: PolicyAction
    allow_model_call: bool
    allow_freeform_response: bool
    allow_ticket_creation: bool
    reason_code: str
    response_key: str | None = None
```

ترتيب القواعد مهم

- افحص السياسة الأعلى أولوية أولاً، مثل منع إجراء غير مصرح.
- سياسة السعر تسبق explain differently حتى لا يخرج سعر حر.
- التوضيح يزيد فقط مع intent مناسب وعدم وجود تقدم.
- لا تعرض OFFER_HUMAN_HANDOFF ما لم توجد قناة بشرية فعلية.
- استخدم reason_code ثابتاً للمراقبة بدل تحليل النص العربي.

سلوك الواجهة	المعنى	Action
سؤال أو رد عادي	تابع المسار الطبيعي	CONTINUE
مثال وسؤال أبسط	المعنى تكرر أو المستخدم مرتبك	EXPLAIN_DIFFERENTLY
شرح السياسة دون رقم	السعر يحدده القسم	APPLY_PRICING_POLICY
زر أو قناة موجودة فعلاً	المسار الآلي لم يكف	OFFER_HUMAN_HANDOFF
429 أو رسالة انتظار صادقة	قرار حماية زمني	TEMPORARILY_SUSPEND

12

الشرح ليس القرار: Response Builder

نصوص واضحة مبنية على الحالة والسياسة

يحول القرار والمرحلة والمتطلب التالي إلى لغة مناسبة. لا يقرر خدمة جديدة ولا يكتب `response_builder.py` قاعدة بيانات. عندما تكون السياسة EXPLAIN_DIFFERENTLY يجب أن يعرف المفتاح النشاط ويقدم شرحا خاصا به.

خريطة شرح سياقية منفصلة عن كتالوج السؤال الرسمي

```

REQUIREMENT_GUIDANCE = {
    "agent_goal": (
        "What work should the agent do instead of an employee? "
        "Examples: support, booking, order tracking."
    ),
    "deployment_channel": (
        "Where will people use it: website, app, or internal system?"
    ),
}

def requirement_guidance(requirement):
    return REQUIREMENT_GUIDANCE.get(
        requirement.key,
        generic_guidance(requirement.value_kind),
    )

```

خصائص الرد الجيد

- يجب أولاً ثم يطلب الخطوة التالية.
- لا يكرر الجملة نفسها عند طلب التبسيط.
- يعرض أمثلة قليلة غير ملزمة.
- لا يدعي سعرا أو موعدا أو قناة بشرية غير موجودة.
- يعرض المسودة بإصدار واضح ويذكر ما يمكن تعديله.
- يفصل رسالة الخطأ التقنية عن النص المناسب للعميل.

الهوية والمقاطع

ضع ردود الهوية والتحية والنطاق كـ `controlled responses`. أجب عنها دون تغيير الخدمة الحالية، ثم اسمح للجلسة بالعودة إلى السؤال المنتظر.

13

المنسق فقط Workflow Graph

العقد قصيرة والمسارات قابلة للرؤية

واحدًا وتعيد تحديثات محددة. لا تنقل كل المنطق Use Case يربط المكونات. العقدة الجيدة تستدعي workflow.py إلى graph؛ وإلا يصبح الاختبار صعبًا وتتعدى إعادة استخدام المكونات خارج LangGraph.

خريطة منطقية قبل كتابة StateGraph

```
START
-> check_ingress
-> classify_message
-> guard_decision
-> route_by_intent

route_by_intent:
  service_question -> answer_knowledge
  ticket_edit      -> validate_and_apply_edit
  ticket_confirm   -> confirm_ticket
  default          -> collect_requirements

collect_requirements
-> evaluate_business_policy
-> build_response
-> END
```

قواعد العقد

- اسم العقدة فعل واضح: classify_message لا process_everything.
- لا تخف الأخطاء؛ ترجمها عند حدود API أو Adapter.
- اجعل route functions حتمية وقصيرة.
- لا تعدل كائن الحالة نفسه في عدة عقد دون نسخ أو عقد واضح.
- سجل run name وtags وmetadata دون بيانات حساسة.

اختبار المسار

اختبر graph بموديلات FakeListChatModel ومستودعات InMemory. ثم نفذ إثباتًا واحدًا على المزود وقاعدة البيانات الحقيقيين.

3

المعرفة والأفعال والبيانات

إضافة RAG والوكلاء المتخصصين والأدوات والتذاكر والتخزين دون ربط منطق العمل
بمزود واحد.

قاعدة المعرفة وRAG الموثق

استرجع أولاً ثم أجب من المصادر المسترجعة فقط
عندما يحتاج الوكيل معلومات الشركة، أنشئ `infrastructure/knowledge_base.py` للاسترجاع
و `application/knowledge_answerer.py` لبناء الإجابة. افصل المعرفة عن Prompt حتى يمكن تحديثها والبحث
فيها وإظهار مصادرها.

العقد الأساسية

مخرجاته	مدخلاته	المكون
مقاطع بمصدر وعنوان ونص	query و service filter و limit	KnowledgeBase.search
answer_ar + source_ids + sufficient_context	سؤال + المقاطع المسترجعة	KnowledgeAnswerer
المصادر الموجودة ضمن retrieved فقط	من النموذج source_ids	Source Guard

لا تقبل مصدراً اخترعه النموذج

```
retrieved = knowledge_base.search(question, service_key)
answer = await answerer.ainvoke({
    "question": question,
    "context": serialize(retrieved),
})

allowed = {item.source_id for item in retrieved}
if not set(answer.source_ids).issubset(allowed):
    return insufficient_context_response()

return KnowledgeAnswerResult(
    text=answer.answer_ar,
    sources=[item for item in retrieved
              if item.source_id in answer.source_ids],
    sufficient_context=answer.sufficient_context,
)
```

متى تستخدم RAG؟

- عندما تتغير المعلومات دون إعادة تدريب النموذج.
- عندما تحتاج ذكر مصدر أو سياسة داخلية.
- عندما توجد أقسام وصلاحيات وصول مختلفة.
- عندما تريد فصل معرفة الشركة عن سلوك الحوار.

لا تستخدمه لكل شيء

قواعد مثل لا تقدم سعراً نهائياً مكانها Policy Engine، لا وثيقة RAG فقط. المعرفة تشرح، والسياسة تمنع أو تسمح.

15

وكلاء الأقسام وسجل المهارات

تخصص منصبت لا مجموعة Prompts مبعثرة

إذا كانت الشركة متعددة الأقسام، أنشئ DepartmentAgentPort وعقد طلب ونتيجة، ثم Registry يربط القسم بال Adapter. سجل المهارات يحدد الخدمات والمصادر والقدرات التي يجوز لكل قسم استخدامها.

عقد ثابت مهما تغير تنفيذ القسم

```
class DepartmentAgentPort(Protocol):
    async def run(
        self,
        request: DepartmentAgentRequest,
    ) -> DepartmentAgentResult: ...

class DepartmentAgentRegistry:
    def __init__(self, agents):
        self._agents = agents

    def get(self, department: Department):
        return self._agents[department]
```

Skill definition محتوى

الحقل	الغرض	مثال
department	هوية المالك	support
service_keys	الخدمات المسموح بها	hosting, monitoring
knowledge_sources	المصادر التي يقرأها	support-policies
tools	الأدوات المصرح بها	ticket_lookup
response_rules	قيود العرض	لا يكشف ملاحظات داخلية

في التكوين Fail closed

عند بدء التطبيق، تحقق أن كل قسم وخدمة مغطاة وأن كل مصدر وأداة معروفان. فشل الإعداد أفضل من وكيل يعمل بنصف صلاحيات مجهولة.

الأدوات: Ports و Adapters قبل التنفيذ

النموذج يطلب فعلا، والتطبيق يقرر وينفذ
عندما يحتاج الوكيل إلى CRM أو تقويم أو ERP أو بريد أو بحث، لا تمنح النموذج اتصالا مباشرا. عرف Port في application/ports ثم Adapter في infrastructure. و idempotency وضع التحقق والصلاحيّة و.

منطق العمل يعتمد على Port لا SDK خارجي

```
class BookingPort(Protocol):
    async def list_slots(self, request: SlotRequest) -> list[Slot]: ...
    async def create_once(
        self,
        idempotency_key: str,
        booking: Booking,
    ) -> BookingWriteResult: ...

class CalendarBookingAdapter(BookingPort):
    # External provider details live here.
    ...
```

دورة تنفيذ أداة

1 اقتراح Tool Call

النموذج يعيد اسما ومعاملات منظمة

2 Schema validation

يتحقق من الأنواع والحدود Pydantic

3 Authorization

هل هذا المستخدم وهذا القسم مسموحان؟

4 Business policy

هل يحتاج تأكيداً أو مراجعة؟

5 Execute once

Adapter مع idempotency و timeout

6 Audit and explain

سجل النتيجة ورد واضح

نوع الأداة	المخاطر	الصيغ
قراءة	كشف بيانات غير مصرح بها	فلتر حسب tenant والدور
كتابة	تكرار أو قيمة خاطئة	تأكيد + validation + idempotency

نوع الأداة	المخاطر	الضبط
إرسال	رسالة إلى جهة خاطئة	تأكيد المستلم ومعاينة
حذف	فقد بيانات	موافقة صريحة وسجل تدقيق

17

التذاكر والمسودة والتأكيد

افصل التحضير عن الأثر النهائي

أي إجراء مهم يستفيد من مرحلتين: Draft قابل للمراجعة، ثم Confirm لإصدار محدد. لا تجعل عبارة موافق تنفذ مسودة قديمة بعد تعديلها. أرسل draft_version في طلب التأكيد.

الإصدار جزء من عقد التأكيد

```
class TicketDraft(BaseModel):
    draft_id: UUID
    version: int = 1
    service_key: ServiceKey
    primary_department: Department
    requirements: dict[str, str]
    additional_features: list[str] = []
    confirmed_version: int | None = None

class ChatRequest(BaseModel):
    message: str
    session_id: str | None = None
    draft_version: int | None = None
```

قواعد تعديل المسودة

- كل تعديل مقبول يزيد version ويلغي confirmed_version.
- المدقق يعيد عمليات منظمة: add/remove/replace/note.
- لا تخزن اسم الحقل مثل additional_features بدل نص الميزة الفعلي.
- التعديل المكرر لا يصنع إصدارا بلا تغيير.
- التعديل غير المرتبط أو غير المسموح يعاد برد مضبوط دون تغيير المسودة.

Idempotency

ابن المفتاح من tenant + draft_id + version أو قيمة مستقرة مكافئة. Repository ينفذ create_once ذريا ويعيد العنصر السابق عند التكرار. في PostgreSQL استخدم unique constraint و ON CONFLICT DO NOTHING، لا فحصا ثم إدخالا منفصلين.

اختبار الإثبات

أنشئ Draft v1، عدله إلى v2، أكد v2 مرتين، ثم تحقق من صف واحد ورقم واحد وسجل created واحد وحالة جلسة ticket_created.

التخزين الدائم والتزامن

للحالة السريعة Redis للحقيقة و PostgreSQL
ابداً بمستودعات InMemory لاختبارات الوحدة، ثم نفذ نفس Ports على PostgreSQL. لا تجعل التطبيق يعرف نوع المستودع؛ dependencies تختار Adapter حسب Settings.

السبب	المخزن	البيانات
استثناء وتزامن ومراجعة	أو مخزن دائم PostgreSQL	جلسة المحادثة
معاملة وقيد فريد وسجل	PostgreSQL	التذكرة والحجز
عدادات زمنية ذرية وسريعة	Redis	Rate limit
تقليل زمن الاسترجاع	اختياري Redis	معرفة Cache
نسخ وإصدارات وصلاحيات	مخزن وثائق	ملفات الشركة

التبديل في composition root فقط

```
@lru_cache
def provide_session_repository() -> SessionRepository:
    settings = get_settings()
    if settings.persistence_backend == "postgresql":
        return PostgresSessionRepository(
            provide_database().sessions,
            settings.tenant_id,
        )
    return InMemorySessionStore()
```

Optimistic concurrency

احفظ revision في الجلسة. عند save استخدم WHERE revision = old_revision ثم زدها. إذا لم يتأثر صف واحد، أعد 409 واطلب إعادة المحاولة بدل فقد تحديث أحد الطلبين.

Tenant isolation

كل استعلام مؤسسي يجب أن يفلتر tenant_id من الخادم، لا من قيمة يرسلها العميل. القيد الفريد نفسه يضم tenant عند الحاجة.

استئناف السياق والتحويل البشري

المقاطعة لا تعني خسارة المهمة

الحوار الواقعي غير خطي. العميل قد يسأل عن الهوية أو السعر أو الأمان ثم يعود. controlled intents تجيب عن المقاطعة، بينما الحالة المنظمة تحفظ current_service و last_question_key. بعد الرد يستمر المسار من آخر نقطة.

متى تنشئ Human Handoff؟

- عندما توجد قناة فعلية ومالك واضح ووقت استجابة معروف.
- عندما يفشل التوضيح الحقيقي عدة مرات بلا تقدم.
- عندما تتطلب السياسة قراراً أو صلاحية بشرية.
- عندما ينخفض اليقين مع أثر مالي أو قانوني أو تشغيلي مهم.

إن لم تكن موجودة	إن كانت القناة موجودة
لا تقل تم التحويل	أنشئ handoff record وأظهر رقما وحالة
قدم مسارا بديلا صادقا	مرر ملخصا ومصادر ومتطلبات
قل إن الربط غير متاح حاليا	حدد القسم والأولوية

لا تبين CAPTCHA وهميا

إذا لم توجد واجهة تحقق ونقطة API لتأكيدتها فلا تقل أكمل التحقق البشري. استخدم Redis و 429 للحماية الحقيقية، واجعل نص الواجهة مطابقا لما تم تنفيذه.

4

الموثوقية والأمان والاختبار

التعامل مع الأعطال والحماية والصلاحيات والتقييم حتى يصبح السلوك قابلاً للإثبات لا مجرد عرض تجريبي.

هندسة ال Prompt والأدلة

التعليمات تكمل العقود ولا تستبدلها

المصنف يجب أن يحدد المسؤولية والعقد وقواعد الرسالة الحالية وأمثلة الحدود. لا تحشو فيه تاريخ الشركة Prompt كاملاً. مرر catalog context من مصدر منظم وformat instructions من Pydantic parser.

بنية Prompt جيدة

- الدور: صنف واستخرج ولا تجب العميل.
- المصادر المسموحة: الكتالوج والمفاتيح الرسمية.
- قاعدة الرسالة الحالية: لا تنقل حقائق من السياق السابق.
- مقطع متصل حرفياً من رسالة العميل: Evidence.
- حالات الإصلاح: لم أفهم، جواب السؤال النشط، تعديل المسودة.
- في النهاية parser تعليمات: Schema.

مختصر مرتبط بعقد Prompt

```
SYSTEM:
You classify and extract. Never answer the customer.
Use only allowed service and requirement keys.
Every extracted requirement must include one contiguous
verbatim evidence substring from the current message.
Do not carry facts from previous context.
```

```
HUMAN:
Conversation context: {conversation_context}
Validation feedback: {validation_feedback}
Current message: {message}
```

واحد ذكي Retry

إذا فشل JSON أو evidence، أرسل feedback محدداً لمحاولة ثانية. احتفظ بالحقائق الصحيحة من المحاولة الأولى وادمج التصحيحات المؤرصة فقط. لا تكرر بلا حد ولا تملك دالتين باسم classify_message لأن الثانية تلغي الأولى بصمت.

Prompt injection

عامل رسالة العميل كبيانات، لا تسمح لتعليمات داخلها بتغيير النظام أو كشف Prompt أو توسيع الأدوات. الحماية الحقيقية تأتي من العقود والصلاحيات، لا من جملة تجاهل التعليمات فقط.

أعطال مزود النموذج

للمستخدم Traceback ليس Timeout

حتى أفضل نموذج يتعطل أو يتأخر أو يعيد JSON غير صالح. أنشئ أخطاء تطبيق مثل MessageClassificationError و TicketFeatureValidationError، ثم ترجم أخطاء المزود عند API 502 ورسالة ثابتة دون كشف stack trace أو مفتاح أو عنوان داخلي.

الاستجابة	المكان الذي ينقطه	ال فشل
502 ومطالبة بإعادة المحاولة	Adapter أو API boundary	ConnectTimeout
محاولة تصحيح واحدة ثم 502	classifier retry/parser	Invalid JSON
503 تخزين غير متاح	Repository	Database unavailable
أو 503 حسب الإعداد fail-open	TrafficGuard	Redis unavailable
409 وإعادة إرسال	Session repository	Write conflict

ترجمة الأخطاء في حدود HTTP

```
try:
    result = await workflow.ainvoke(payload)
except (MessageClassificationError, LangChainException) as exc:
    raise HTTPException(
        status_code=502,
        detail="The AI provider could not process the message",
    ) from exc
except SessionWriteConflictError as exc:
    raise HTTPException(
        status_code=409,
        detail="The conversation changed; retry this message",
    ) from exc
```

Timeout budgets

- حدد timeout للمحاولة و timeout إجمالي للسلسلة.
- أعد المحاولة فقط للأخطاء المؤقتة وبعدد صغير.
- لا تعيد أداة كتابة دون idempotency.
- سجل نوع الخطأ والمدة والمزود دون محتوى حساس.
- اجعل تغيير النموذج إعداداً، لكن لا تحذف معالجة الأعطال عند التغيير.

والمصادقة والصلاحيات Traffic Guard

ثلاث مشكلات مختلفة تحتاج ثلاث طبقات

يجيب: ماذا يحق له؟ لا Authorization يجب: من المستخدم؟ Authentication يجب: كم طلباً؟ Rate limiting
تخلطها مع semantic repetition أو مع Prompt.

الطبقة	الملف	مثال قرار
Traffic	infrastructure/traffic_guard.py	أكثر من الحد داخل 60 ثانية - < 429
Authentication	auth service أو middleware	صالح ومنتهي الصلاحية Cookie/JWT
Authorization	policy/dependency	موظف TXSaaS يرى تذاكر قسمه فقط
Semantic	application/policy_engine.py	كرر الهدف -> اشرح بطريقة أخرى

صحيح Traffic Guard

- مفتاح جلسة ومفتاح عميل منفصلان.
- نافذة زمنية حقيقية وعمليات Redis ذرية.
- في Retry-After 429.
- سياسة fail-open معلنة عند تعطل Redis.
- لا تستخدم معنى الرسالة كدليل سرعة.

حماية لوحة التذاكر

احم الصفحة وAPI الذي يغذيها؛ حماية tickets/ في الواجهة وحدها لا تكفي. استخدم Cookie HttpOnly موقع، انتهاء زمني، logout يحذفها، ويفضل أن يتحقق Backend أيضاً من هوية الموظف ودوره قبل إرجاع البيانات.

أقل صلاحية

لا ترسل كل التذاكر ثم تخفها بالواجهة. الاستعلام نفسه يفلتر tenant والقسم والدور في الخادم.

استراتيجية الاختبار

اثبت العقود والرحلة والأنظمة الخارجية

اختبارات الوكيل ليست سؤالاً واحداً للنموذج الحقيقي. اجعل معظمها حتمياً بموديلات وهمية وAdapters ذاكرة، ثم أضف اختبارات تكامل محدودة للمزود PostgreSQL وRedis والواجهة.

مثال	ما يختبر	المستوى
لم أفهم -> clarification	دالة أو Guard أو Policy	Unit
متصل evidenceمذكور و website_goal	Schema و Prompt	Contract
طلب ثم جواب ثم Draft	المسارات والحالة	Workflow
يعيد صفا واحدا create_once	التخزين والتزامن	Repository
409 و 429 و 502 و 503	والأخطاء HTTP	API
API -> PostgreSQL -> query	المكونات الحقيقية	Live proof
دقة التوجيه واكتمال المتطلبات	جودة دلالية على Dataset	Evaluation

اختبار Guard

لكل قاعدة Positive و Negative case

```
def test_confusion_is_repaired():
    result = guard_message_decision(
        conversation_with_active_question(),
        model_decision(intent="unknown"),
        "I did not understand; explain simply",
    )
    assert result.primary_intent is MessageIntent.CLARIFICATION
    assert result.extracted_requirements == []

def test_real_edit_is_not_confusion():
    result = guard_message_decision(
        draft_conversation(),
        model_decision(intent="service_request"),
        "Add email notifications",
    )
    assert result.primary_intent is MessageIntent.TICKET_EDIT
```

لا تثبت الخطأ في الاختبار

إذا أصلحت سلوكاً مثل fake human check، حدث الاختبارات التي كانت تتوقعه. اسم الاختبار وتوقعه يجب أن يصف النتيجة المطلوبة للعميل، لا تفاصيل داخلية قديمة مثل عدم وجود classification.

الترميز

في ملفات عربية استخدم UTF-8 صراحة. لا تقرأ ملفاً بـ Get-Content الافتراضي ثم تعيد حفظه بترميز آخر؛ وإلا تتحول العربية إلى رموز Ø ولأ وتفشل الاختبارات رغم صحة التطبيق.

المراقبة والتقييم المستمر

اعرف لماذا أخطأ النظام دون كشف بيانات العميل

المراقبة الجيدة تربط كل رسالة بـ session hash و stage و service و policy action و latency و error type. لا تحتاج تسجيل النص الخام دائماً، خصوصاً إذا كان حساساً. استخدم tracing مع إخفاء المدخلات والمخرجات عند الحاجة.

حقول مفيدة

الحقول	الفئة
الغام session hash و tenant_id و session_id	هوية آمنة
stage و intent و service و next_requirement	تدفق
action و reason_code	سياسة
latency و attempts و model و provider	نموذج
tool_name و result status و idempotency outcome	أداة
grounded sources و clarification count و task success	جودة

انحدار Dataset

- اجمع أمثلة واقعية بعد إزالة البيانات الحساسة.
- غط اللهجات والأخطاء الإملائية والرسائل المركبة.
- ضع expected intent و service و requirements و policy.
- أعد تشغيل المجموعة عند تغيير Prompt أو نموذج أو كتالوج.
- قارن الجودة والتكلفة والزمن قبل اعتماد التغيير.

متى تغير النموذج؟

غير النموذج عندما تثبت البيانات أن الدقة أو الزمن أو التكلفة تحسنت. إذا كانت المشكلة في Guard أو State أو Policy فلن يحلها نموذج أكبر بصورة موثوقة.

5

إعادة الاستخدام والتوسع

وصفات جاهزة لإضافة الخصائص وتكييف الهيكل لوكلاء شركات ومهام مختلفة وخطة تنفيذ تدريجية.

وصفات إضافة الخصائص

كل حاجة جديدة لها مكون وعقد واختبار قبل إنشاء ملف جديد أسأل: هل هذه حقيقة مجال، قرار تطبيق، تكامل خارجي، أم طريقة عرض؟ ضعها في الطبقة الموافقة. الوصفات التالية ليست أسماء إلزامية، لكنها حدود مفيدة.

إذا أردت تصحيح قرارات النموذج

اختبر	لا تصنع فيه	ضع فيه	أنشئ
حالة صحيحة وأخرى مشابهة لا تنطبق	أو رد نهائي DB أو LLM	وقواعد intent repair و grounding واضحة	application/decision_guard.py

إذا أردت فهم التكرار ومحاولات التوضيح

اختبر	لا تصنع فيه	ضع فيه	أنشئ
تكرار خدمة لا يصبح هجوماً؛ تقدم يعيد العداد	معدل طلبات أو حظر IP	وعداد توضيح semantic signature وقواعد reset	application/policy_engine.py

إذا أردت أداة تنفذ فعلاً

اختبر	لا تصنع فيه	ضع فيه	أنشئ
Fake adapter ثم sandbox idempotency و حقيقي	قرار صلاحية داخل SDK	عقد ومعاملات وتكامل خارجي	application/ports/x.py + infrastructure/x_adapter.py

إذا أردت معرفة موثقة

اختبر	لا تصنع فيه	ضع فيه	أنشئ
يرفض source_id غير مسترجع	قواعد منع تجارية فقط داخل وثيقة	استرجاع ومصادر وعقد إجابة	knowledge_base.py + knowledge_answerer.py

مقياس جودة التقسيم

إذا احتجت تشغيل نموذج أو Redis أو PostgreSQL لاختبار دالة Guard أو Policy، فحدود الطبقات على الأغلب غير صحيحة.

تكيف الهيكل لوكلاء مختلفين

الهيكل ثابت نسبياً وما يتغير هو المجال والسياسة والأدوات
لا تتنسخ وكيل خدمة العملاء وتغير الاسم فقط. احتفظ بالطبقات، ثم أعد تعريف outcome و catalog و state و policies و tools و tests حسب المهمة.

نوع الوكيل	الحالة الأساسية	الأدوات	السياسات الحرجة
خدمة عملاء	الخدمة والمتطلبات والتذكيرة	إنشاء ومتابعة تذكيرة	سعر وتحويل وقسم
مبيعات	Lead stage واحتياجات وميزانية	و موعد CRM	عدم اختراع عرض أو خصم
حجوزات	خدمة وموعد وعمل وتأكيـد	تقويم ودفع اختياري	توفر و idempotency وإلغاء
دعم فني	نظام وعرض وخطوات ودرجة أثر	Knowledge و incident	عدم تنفيذ تغيير حساس دون موافقة
معرفة داخلية	دور المستخدم وسؤال ومصادر	بحث وثائق	صلاحيات ومصادر فقط
موارد بشرية	هوية موظف ونوع طلب	وتذاكر داخلية HRIS	خصوصية وفصل أدوار ومراجعة بشرية
عمليات	وحالة وموافقات Job	ERP و Queues	حدود تنفيذ وتدقيق واسترجاع

أسئلة إعادة التصميم

- ما الكيان النهائي: Ticket أم Booking أم Lead أم Incident؟
- ما مراحل الحالة الرسمية؟
- ما الحقائق المطلوبة قبل التنفيذ؟
- ما الذي يقرأ فقط وما الذي يكتب؟
- من يملك الموافقة وما الذي يجب تدقيقه؟
- ما البيانات المقيمة وكيف تفصل tenant والدور؟
- ما اختبارات الضرر: تكرار، مستلم خاطئ، قيمة قديمة، صلاحية ناقصة؟

المزود ليس الهوية

يمكن تبديل OpenRouter أو DeepSeek أو غيرهما داخل model adapter إذا كان العقد متوافقاً. لا تجعل اسم المزود منتشرًا في Domain و Policy و Workflow.

خطة تنفيذ مشروع جديد

مراحل صغيرة لكل منها دليل انتهاء

- 1 **تحليل العمل**
وحدود وخدمات وسياسات وعشر محادثات Outcome
- 2 **عقود Domain**
Catalog و Decision و State و Enums
- 3 **نواة حتمية**
Response و Policy و Guard و Collector
- 4 **Workflow و API**
مسار واحد كامل مع Memory adapters
- 5 **معرفة وأدوات**
حسب الحاجة Ports و RAG
- 6 **أثر مؤكد**
Audit و Idempotency و Confirm و Draft
- 7 **تخزين وحماية**
وإثبات حقيقي Redis و PostgreSQL
- 8 **واجهة وصلاحيات**
تجربة مستخدم ولوحة داخلية محمية
- 9 **تقييم وتحسين**
Live dialogue و Tracing و Dataset

Definition of Done لكل مرحلة

المرحلة	لا تعتبرها منتهية حتى
العقود	تفشل القيم غير المسموحة وتنتج الأمثلة الصحيحة
الحوار	جلسة متعددة الرسائل تكمل المتطلبات ولا تختزع
المعرفة	الإجابة تعرض مصادر مسترجعة وترفض مصدرا مختلفا
الأداة	التنفيذ المكرر لا يكرر الأثر
التخزين	تثبت الصفوف والحالة والتدقيق من قاعدة حقيقية
الواجهة	تعرض الأخطاء والحالات والتحميل دون كشف أسرار

لا توسع قبل إثبات المسار

ابن خدمة واحدة كاملة من الرسالة حتى الأثر والتخزين، ثم عمم النمط. عشر خدمات نصف مكتملة أصعب في التصحيح من خدمة واحدة مثبتة.

الأخطاء المعمارية الشائعة

علامات تخبرك أن المشروع يحتاج إعادة فصل

الإصلاح	النتيجة	الخطأ
عقود و Policy و Guard وأدوات منفصلة	سلوك هش وغير قابل للاختبار	ضخم يفعل كل شيء Prompt
عقد قصيرة و Use Cases	صعوبة فهم المسار	بعقدة واحدة Workflow
وحدود اعتماد Ports	اقتران وتكلفة اختبار	يستورد Domain FastAPI/SQLAlchemy
فصل semantic عن traffic	خطر عملاء مرتبكين	كل تكرار هجوم
تحقق وصلاحيه على الخادم	مكتشوف API	الحماية في الواجهة فقط
Draft + version + confirm	أثر خاطئ أو مكرر	تنفيذ قبل التأكد
فيد فريد و create_once ذري	Race condition	فحص ثم إدخال
اختبر نتيجة العميل والعقد	ثبيت الأخطاء	اختبارات على تفاصيل قديمة
بحث AST أو rg ومراجعة	إلغاء صامت للتعريف الأول	تعريف دالة مرتين
تعريف واحد واختبار import	قيمة تخفي قيمة	إعدادات مكررة

متى تنشئ ملفاً جديداً؟

- عندما يوجد مفهوم له عقد مستقل واختبارات مستقلة.
- عندما تختلف أسباب التغيير: سياسة السعر لا تتغير مع Redis.
- عندما يوجد Adapter آخر لنفس Port.
- عندما تجاوزت الدالة مسؤولية واحدة وأصبحت تحتاج بيانات لا تخصها.

لا تفرط في التقسيم

ملف لكل دالة ليس هدفاً. اجمع الوظائف التي تتغير للسبب نفسه، وافصل الوظائف التي تحتاج أسباباً واختبارات وتبعيات مختلفة.

قالب تصميم خاصية جديدة

ورقة قرار تملؤها قبل التنفيذ

السؤال	ما الذي نكتبه
ما المشكلة؟	سلوك واحد يمكن ملاحظته
من يطلبها؟	مستخدم ودور وtenant
ما المدخل؟	Pydantic model وحدود وقيم
ما المخرج؟	PolicyDecision أو Result model
هل لها أثر؟	قراءة أم كتابة أم إرسال أم حذف
ما الصلاحية؟	شرط الدور والملكية والموافقة
أين تسكن؟	API أو Infrastructure أو Application أو Domain
ما الفشل؟	unavailable وconflict وinvalid input وTimeout
كيف تمنع التكرار؟	أو لا حاجة idempotency key
كيف تختبر؟	وحدة وتكامل وحالة سلبية وإثبات حي

مثال بطاقة خاصية قابلة للتنفيذ

```
FEATURE: create_booking
OUTCOME: one confirmed booking
INPUT: BookingDraft + confirmed_version
OUTPUT: BookingWriteResult
POLICY: authenticated user; slot still available
PORT: BookingRepository.create_once
ADAPTER: PostgreSQLBookingRepository
IDEMPOTENCY: tenant + draft_id + version
ERRORS: conflict=409, storage=503
TESTS:
- rejects stale version
- duplicate confirm returns same booking
- database contains one row
- audit contains created event
```

ابدأ من الاختبار

إذا لم تستطع كتابة توقع واضح للخاصية، لا تبدأ في كتابة Adapter أو Prompt. عد إلى تعريف النتيجة والسياسة.

دليل الخصائص: ماذا أنشئ لكل حاجة؟

خريطة سريعة من الحاجة إلى الملف والاختبار

استخدم هذا الجدول عندما تظهر خاصية جديدة. ابدأ من الحاجة، ثم أنشئ طبقة واحدة بمسؤولية محددة، وحدد عقد المدخلات والمخرجات قبل كتابة التنفيذ.

اختبار القبول	المسؤولية	الملف أو المكون	الحاجة
intent/service صحيح و JSON متوقعان	استدعاء النموذج وتحويله إلى MessageDecision	message_classifier.py	تصنيف الرسائل
يحذف الدليل المختلف ولا يغير حالة صحيحة	وإصلاح قواعد حتمية Grounding	decision_guard.py	تصحيح قرار غير موثوق
كل خدمة لها قسم ومفاتيح فريدة	خدمات وأقسام ومتطلبات وأنواع قيم	service_catalog.py	تعريف خدمات
جواب السؤال النشط يقدم المرحلة	تطبيق الحقائق وحساب النواقص	requirements_collector.py	جمع متطلبات
التقدم بعيد العدادات ولا يحدث خطر	توقيع دلالي وشرح مختلف	policy_engine.py	تكرار المعنى
طلب خدمة مكرر لا يعد فشل توضيح	عد confusion الحقيقي فقط	policy_engine.py	محاولات التوضيح
Redis مع Retry-After و 429	نافذة زمنية وجلسة وعمل	traffic_guard.py	حد الطلبات
لا يكرر السؤال ولا يدعي ميزة غائبة	شرح حسب next_requirement والسياسة	response_builder.py	ردود سياقية
يرفض source_id غير مسترجع	استرجاع ومصادر وإجابة	knowledge_base + answerer	معرفة موثقة
تغطية كاملة وفشل إعداد واضح	توجيه إلى وكيل متخصص	department_agent + registry	أقسام متعددة
Fake sandbox ثم timeout	عقد نظيف وتكامل مزود	port + adapter	أداة خارجية
يرفض إصدارا قديما	موافقة وإصدار وصلاحيات	policy + confirmation	إجراء حساس
تأكيدان يعيدان نفس الكائن	ذرية Idempotency	repository.create_once	منع التكرار
حدث واحد لكل أثر نهائي	من فعل ماذا ومتى	audit event domain/repository	سجل تدقيق
409 عند write conflict	State و revision	session_repository	حفظ الجلسة
المسار المحمي يرفض بلا Cookie	جلسة موقعة وانتهاء وخروج	auth service/middleware	مصادقة موظف

اختبار القبول	المسؤولية	الملف أو المكون	الحاجة
لا يرى قسم تذاكر قسم آخر	فترة tenant/department/role	authorization dependency	صلاحية قسم
502 بلا traceback	وترجمة خطأ gretry و Timeout	adapter + API mapping	عطل نموذج
مقارنة قبل وبعد تغيير النموذج	حالات ولهجات وتوقعات	evaluations + dataset	تقييم جودة

الخلاصة وقائمة الإطلاق

من الفكرة إلى وكيل موثوق

قاعدة ذهبية

لا تبدأ من اختيار النموذج أو كتابة Prompt. ابدأ من قرار العمل الذي تريد أتمتته، والبيانات المسموح استخدامها، وما الذي يجب أن يحدث عند الشك أو الفشل.

- الهدف التجاري ونطاق الوكيل مكتوبان ويمكن اختبارهما.
- لكل قرار من النموذج عقد Pydantic وطبقة Guard حتمية.
- تكرار المعنى منفصل عن معدل الطلبات والحماية الأمنية.
- المعرفة المسترجعة مقيدة بالمصادر التي تم جلبها فعلا.
- الأدوات خلف Ports واضحة، والصلاحيات قبل التنفيذ لا بعده.
- الحالة محفوظة، والتأكدات الحساسة تستخدم idempotency وسجل تدقيق.
- هناك اختبارات وحدة، تكامل، محادثة متعددة الرسائل، وإثبات حي.
- رسائل الفشل صادقة ولا تدعي وجود تحويل أو تحقق غير مطبق.

إذا نجحت هذه القائمة، تستطيع تغيير الشركة أو المجال أو النموذج دون هدم البنية. الذي يتغير غالبا هو الكنالوج، المعرفة، السياسات، والأدوات؛ أما الطبقات والعقود فتظل قابلة لإعادة الاستخدام.